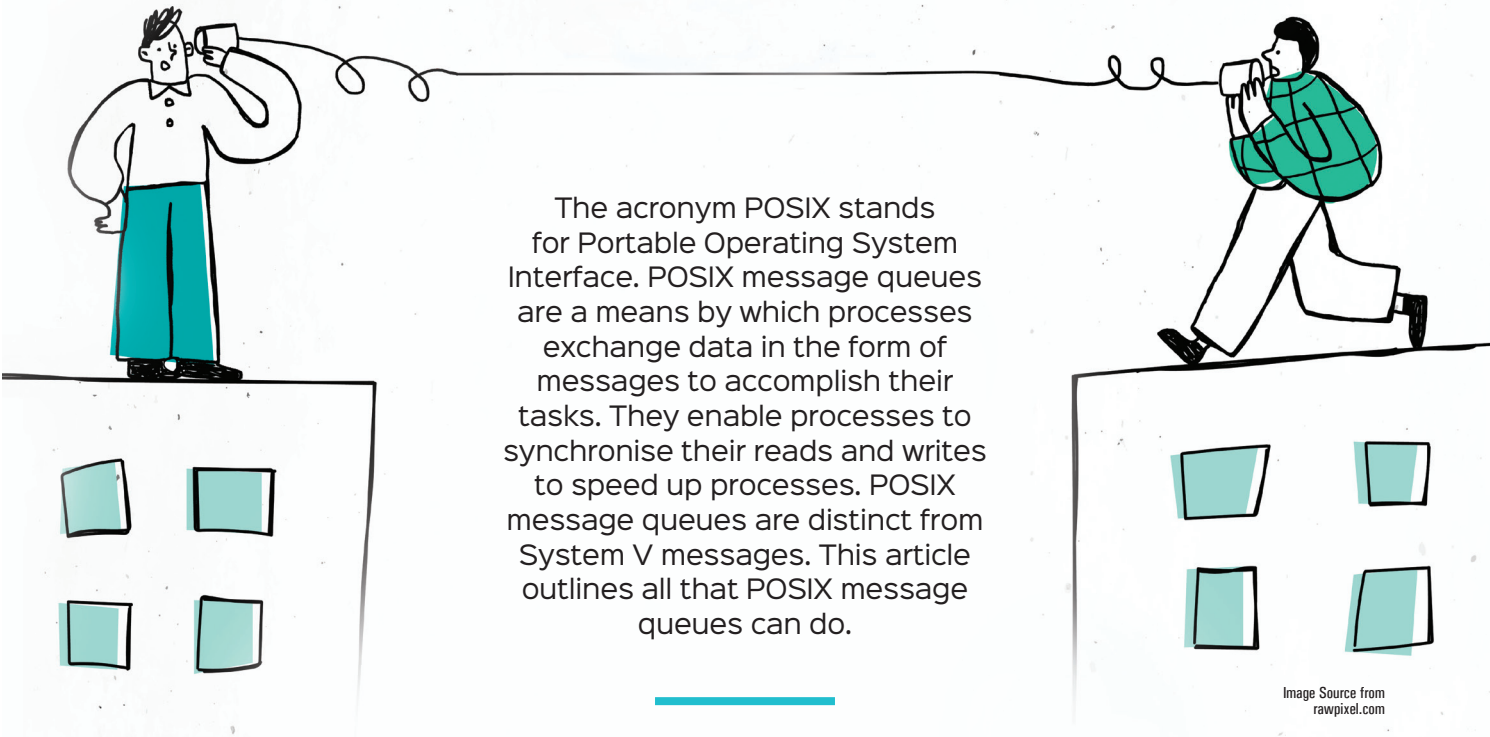
# The Usefulness of POSIX Message Queues



The acronym POSIX stands for Portable Operating System Interface. POSIX message queues are a means by which processes exchange data in the form of messages to accomplish their tasks. They enable processes to synchronise their reads and writes to speed up processes. POSIX message queues are distinct from System V messages. This article outlines all that POSIX message queues can do.

Image Source from rawpixel.com

A program is a set of instructions written to accomplish a certain task — it needs to be executed to perform these specific tasks. A running program, one which is in execution, is called a process. A process contains all the information required by the program to run, such as the code text section, the variables associated with the code, the methods/functions, etc. Modern computers allow multiple processes to run at the same time. Often, in such an environment, processes may need to communicate with each other to accomplish their tasks. A process may depend on another process for some information pertinent to its completion or may want to access the same resources. As an example, two processes may want to read and write to the same shared space, and may need to synchronise their reads and writes. Such an operation requires a tool using which the processes can cooperate. Therefore, there is a need to implement mechanisms for these dependent processes to gain access to the information they require.

Inter Process Communication (IPC) involves the implementation of a set of such mechanisms like message passing or shared memory. IPC mechanisms are an efficient way of handling process synchronisation because they entail speeding up, and are an easy way to share common data and allow code segmentation with thread creation. Linux provides a platform that supports several IPC mechanisms such as pipes, FIFO, message queues, shared memory, semaphore and socket programming. Their usage varies according to the application requirements.

If you want to communicate between related processes, with parent-child relationship, then pipe is a suitable communication mechanism. However, in case we need to communicate between unrelated processes, FIFO can be used. Both the named (pipe) and unnamed (FIFO) pipes are half duplex and of zero buffering capacity. If a developer needs to store message based information and later wants to retrieve it, then we have to use message queues. The problem with message queues is that once you retrieve the message, the message is lost; so messages can't be broadcast and two-way copying increases overhead from user space to kernel space, and vice versa. Shared memory can overcome these problems and is one of the fastest IPC mechanisms, since the information is stored only in the user space. In shared memory, synchronisation between write processes is a big issue, which creates a race condition. So, semaphore is used